

User Registration System - Complete Documentation

Table of Contents

1. [System Overview](#)
 2. [Architecture Components](#)
 3. [Data Flow Diagram \(DFD\)](#)
 4. [Code Deep Dive](#)
 5. [Request-Response Flow](#)
 6. [Validation Rules](#)
 7. [Error Handling](#)
 8. [Security Considerations](#)
 9. [Example Requests](#)
-

System Overview

This is a **Node.js/Express** user registration system that implements:

-  RESTful API endpoint for user registration
-  Joi schema validation middleware
-  Async error handling with asyncHandler
-  Database integration using Mongoose/ORM
-  Custom error response handling

Technology Stack

- **Backend Framework:** Node.js + Express.js
 - **Validation Library:** Joi
 - **Database:** MongoDB (inferred from UserModel)
 - **Error Handling:** Custom ErrorResponse utility
 - **Async Management:** asyncHandler wrapper
-

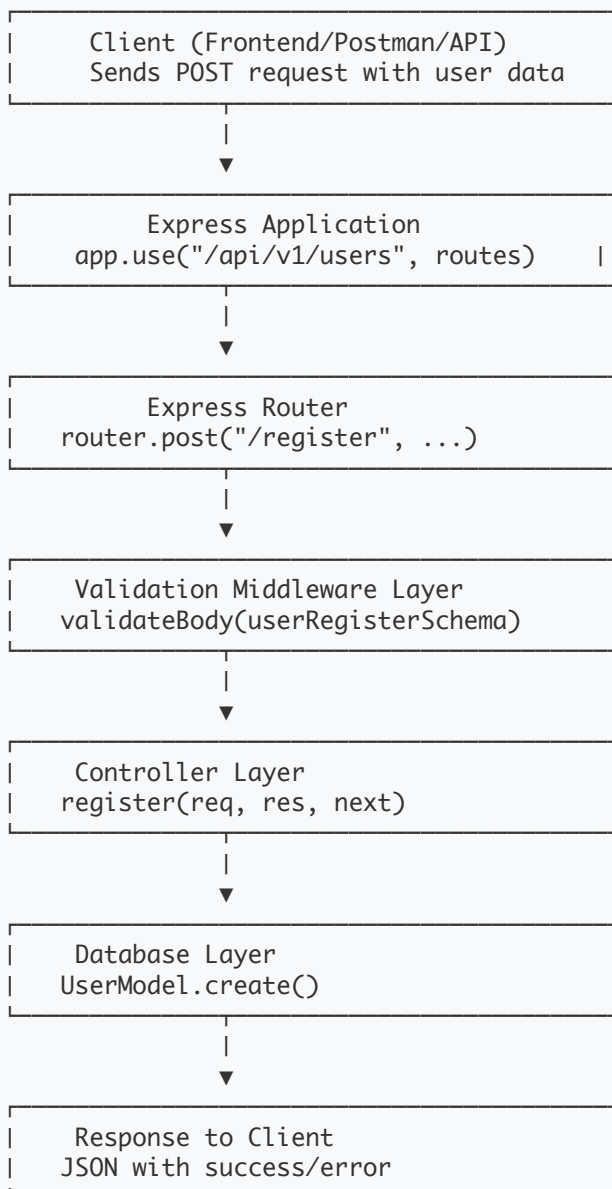
Architecture Components

1. API Endpoint Structure

Protocol: HTTP
Method: POST
Host: localhost
Port: 9000
Base Path: /api/v1
Resource: users
Action: register

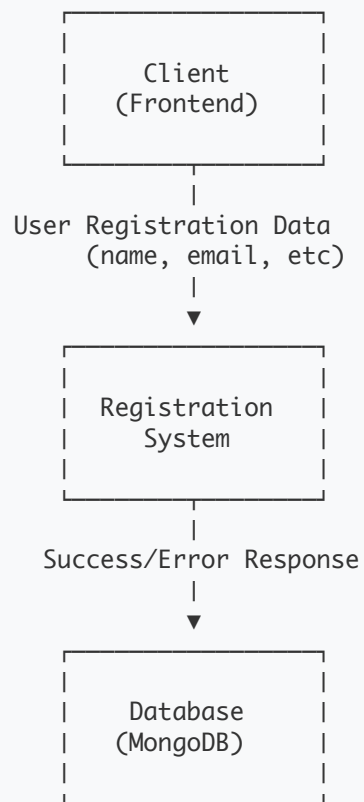
Full Endpoint: POST http://localhost:9000/api/v1/users/register

2. System Layers

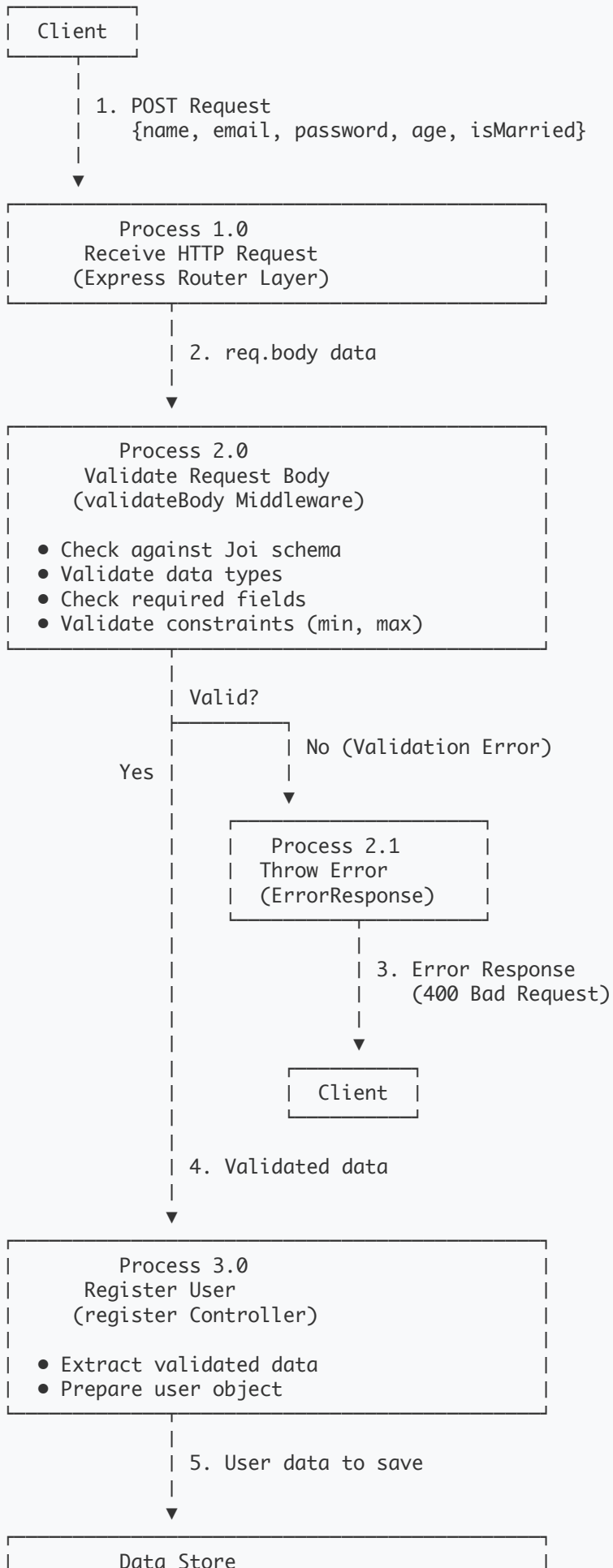


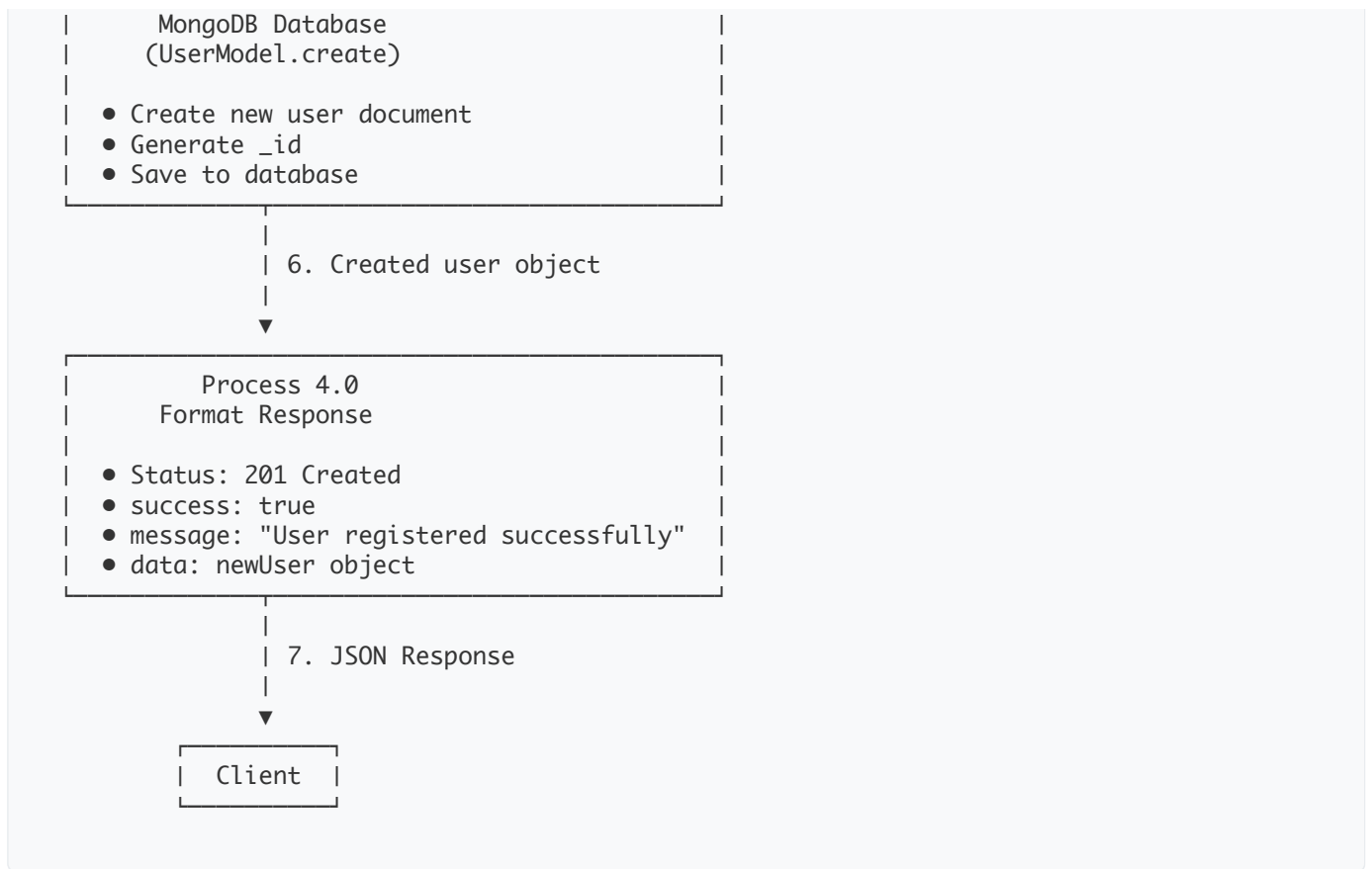
Data Flow Diagram (DFD)

Level 0 DFD (Context Diagram)



Level 1 DFD (Detailed Process)





Data Flow Table

Step	From	To	Data	Description
1	Client	Express Router	HTTP POST + JSON Body	User sends registration data
2	Router	Validation Middleware	req.body	Router passes request to validator
3	Validator	Joi Schema	req.body	Validate against schema rules
4a	Validator	Controller	Validated data	If valid, pass to controller
4b	Validator	Client	Error Response	If invalid, return 400 error
5	Controller	UserModel	User object	Controller prepares data for DB
6	UserModel	MongoDB	Document	Save user to database
7	MongoDB	UserModel	Created document	Return saved user with _id
8	UserModel	Controller	newUser	Return created user object
9	Controller	Client	JSON Response	Send 201 success response

Code Deep Dive

1. Route Configuration

```

app.use("/api/v1/blogs", blogRoutes);

router.post("/register", validateBody(userRegisterSchema), register);

```

Explanation:

Line 1: `app.use("/api/v1/blogs", blogRoutes);`

- **Purpose:** Mounts blog routes at `/api/v1/blogs` path
- **Note:** This line is unrelated to user registration, likely from the same file
- **Functionality:** Any request to `/api/v1/blogs/*` will be handled by `blogRoutes`

Line 2: `router.post("/register", validateBody(userRegisterSchema), register);`

- **Method:** `router.post()` - Defines an HTTP POST endpoint
- **Path:** `"/register"` - The route path (combined with base path becomes `/api/v1/users/register`)
- **Middleware Chain:** Express executes middleware in order:
 1. **First:** `validateBody(userRegisterSchema)` - Validation middleware
 2. **Second:** `register` - Controller function (only if validation passes)

Middleware Chain Flow:

```
Request → validateBody() → register() → Response
           ↓ (if error)
           Error Handler
```

2. Registration Controller

```
export const register = asyncHandler(async (req, res, next) => {
  const { name, age, email, isMarried, password } = req.body;

  let newUser = await UserModel.create({
    name,
    age,
    email,
    isMarried,
    password /* : hashedPassword, */,
  });

  res.status(201).json({
    success: true,
    message: "User registered successfully",
    data: newUser,
  });
});
```

Deep Explanation:

asyncHandler Wrapper

```
export const register = asyncHandler(async (req, res, next) => {
```

- **Purpose:** Wraps async functions to automatically catch errors
- **How it works:**
 - If any error occurs in the async function, it's automatically passed to `next(error)`
 - Eliminates need for try-catch blocks
 - Example implementation:

```
const asyncHandler = (fn) => (req, res, next) => {
  Promise.resolve(fn(req, res, next)).catch(next);
};
```

Destructuring Request Body

```
const { name, age, email, isMarried, password } = req.body;
```

- **Purpose:** Extract validated fields from request body
- **ES6 Destructuring:** Creates individual variables from object properties
- **Before this line:** Data has already been validated by `validateBody` middleware
- **Guaranteed:** At this point, data matches the Joi schema requirements

Example:

```
// req.body = { name: "John", age: 25, email: "john@example.com", password: "pass123" }
// After destructuring:
// name = "John"
// age = 25
// email = "john@example.com"
// password = "pass123"
// isMarried = undefined (optional field)
```

Creating User in Database

```
let newUser = await UserModel.create({
  name,
  age,
  email,
  isMarried,
  password /* : hashedPassword, */,
});
```

- **UserModel.create():** Mongoose method to create and save a new document
- **await:** Pauses execution until database operation completes
- **Object Shorthand:** `{ name, age }` is equivalent to `{ name: name, age: age }`
- **Comment Notice:** `/* : hashedPassword, */` indicates password should be hashed (SECURITY ISSUE - see below)

What happens internally:

1. Creates a new User document
2. Validates against Mongoose schema
3. Generates unique `_id`
4. Saves to MongoDB collection
5. Returns the saved document

Return value (newUser):

```
{
  _id: "507f1f77bcf86cd799439011",
  name: "John Doe",
  age: 25,
  email: "john@example.com",
  isMarried: false,
  password: "pass123", // ⚠️ PLAIN TEXT - SECURITY RISK
  createdAt: "2024-01-15T10:30:00.000Z",
  updatedAt: "2024-01-15T10:30:00.000Z"
}
```

Sending Response

```
res.status(201).json({
  success: true,
  message: "User registered successfully",
  data: newUser,
});
```

- **res.status(201):** Sets HTTP status code to 201 (Created)
 - **200:** OK (general success)

- **201**: Created (resource successfully created)
- **400**: Bad Request
- **500**: Internal Server Error
- **res.json()**: Sends JSON response to client
- **Response Structure:**

```
{
  success: boolean,    // Indicates if operation succeeded
  message: string,      // Human-readable message
  data: object          // The created user object
}
```

Complete Response Example:

```
{
  "success": true,
  "message": "User registered successfully",
  "data": {
    "_id": "507f1f77bcf86cd799439011",
    "name": "John Doe",
    "age": 25,
    "email": "john@example.com",
    "isMarried": false,
    "password": "pass123",
    "createdAt": "2024-01-15T10:30:00.000Z",
    "updatedAt": "2024-01-15T10:30:00.000Z",
    "__v": 0
  }
}
```

3. Validation Middleware

```
import ErrorResponse from "../utils/ErrorResponse.util.js";

export const validateBody = (schema) => {
  return (req, res, next) => {
    let { error, value } = schema.validate(req.body, {
      abortEarly: false,
    });

    if (error) {
      let message = error.details.map((err) => err.message).join(", ");
      throw new ErrorResponse(message, 400);
    }

    req.body = value;
    next();
  };
};
```

Deep Explanation:

Higher-Order Function Pattern

```
export const validateBody = (schema) => {
  return (req, res, next) => {
    // middleware logic
  };
};
```

- **Outer Function:** `validateBody(schema)` - Accepts a Joi schema
- **Inner Function:** `(req, res, next) => {}` - The actual Express middleware
- **Why this pattern?:** Allows us to create reusable validation middleware with different schemas

Usage Example:

```
// We can create multiple validators with different schemas
router.post("/register", validateBody(userRegisterSchema), register);
router.post("/login", validateBody(userLoginSchema), login);
router.post("/update", validateBody(userUpdateSchema), update);
```

Joi Validation

```
let { error, value } = schema.validate(req.body, {
  abortEarly: false,
});
```

- **schema.validate():** Joi method that validates data against schema
- **Parameters:**
 - `req.body`: The data to validate
 - `{ abortEarly: false }`: Options object

abortEarly: false explanation:

- **true** (default): Stops at first validation error
- **false**: Collects ALL validation errors before returning

Example with abortEarly: true:

```
// Input: { email: "invalid", age: 150 }
// Returns only: "email must be a valid email"
```

Example with abortEarly: false:

```
// Input: { email: "invalid", age: 150 }
// Returns: "email must be a valid email, age must be less than or equal to 90"
```

Return value:

```
{
  error: {
    details: [
      { message: "email must be a valid email", path: ["email"], type: "string.email"
    },
      { message: "age must be less than or equal to 90", path: ["age"], type: "number.m
    ax" }
    ],
  },
  value: { /* sanitized/converted values */ }
}
```

Error Handling

```
if (error) {
  let message = error.details.map((err) => err.message).join(", ");
  throw new ErrorResponse(message, 400);
}
```

Breaking it down:

1. Check if validation failed:

```
if (error) {
```

- If `error` exists, validation failed

2. Extract all error messages:

```
let message = error.details.map((err) => err.message).join(", ");
```

Step by step:

```
// error.details is an array:
[
  { message: "email must be a valid email" },
  { message: "age is required" }
]

// .map((err) => err.message) extracts messages:
["email must be a valid email", "age is required"]

// .join(", ") combines with comma:
"email must be a valid email, age is required"
```

3. Throw custom error:

```
throw new ErrorResponse(message, 400);
```

- **ErrorResponse:** Custom error class
- **message:** Combined error messages
- **400:** HTTP status code (Bad Request)
- **throw:** Throws error to be caught by error handling middleware

ErrorResponse class likely looks like:

```
class ErrorResponse extends Error {
  constructor(message, statusCode) {
    super(message);
    this.statusCode = statusCode;
  }
}
```

Success Case

```
req.body = value;
next();
```

- **req.body = value:** Replace request body with sanitized/converted values from Joi
- **Why?:** Joi may convert types (string "25" → number 25) or strip unknown fields
- **next():** Pass control to next middleware (the `register` controller)

Example transformation:

```
// Original req.body (from client):
{
  "name": "John",
  "age": "25",           // String
  "email": " john@example.com ", // Extra spaces
  "password": "pass123",
  "extraField": "hack"   // Unknown field
}

// After validation (req.body = value):
{
  "name": "John",
  "age": 25,             // Converted to number
  "email": "john@example.com", // Trimmed
  "password": "pass123"
  // extraField removed
}
```

4. Joi Validation Schema

```
export const userRegisterSchema = Joi.object({
  name: Joi.string().min(3).max(49).optional(),
  email: Joi.string().email().required(),
  password: Joi.string().required().min(3).max(50),
  age: Joi.number().required().min(1).max(90),
  isMarried: Joi.boolean().optional(),
});
```

Field-by-Field Explanation:

1. name Field

```
name: Joi.string().min(3).max(49).optional()
```

Validation	Rule	Example
Type	<code>string()</code>	Must be a string
Min Length	<code>min(3)</code>	At least 3 characters
Max Length	<code>max(49)</code>	At most 49 characters
Required	<code>optional()</code>	Field can be omitted

Valid Examples:

- `"John"` ✓
- `"John Doe"` ✓
- `"A"` ✗ (too short)
- `"Very Long Name That Exceeds The Maximum Length"` ✗ (too long)
- `undefined` ✓ (optional)

- `null` ❌ (must be string or omitted)

2. email Field

```
email: Joi.string().email().required()
```

Validation	Rule	Example
Type	<code>string()</code>	Must be a string
Format	<code>email()</code>	Must be valid email format
Required	<code>required()</code>	Field must be present

Valid Examples:

- `"user@example.com"` ✅
- `"john.doe@company.co.uk"` ✅
- `"invalid-email"` ❌
- `"@example.com"` ❌
- `undefined` ❌ (required)

Email validation checks:

- Has @ symbol
- Has domain name
- Valid TLD (.com, .org, etc.)
- Proper format before and after @

3. password Field

```
password: Joi.string().required().min(3).max(50)
```

Validation	Rule	Example
Type	<code>string()</code>	Must be a string
Required	<code>required()</code>	Field must be present
Min Length	<code>min(3)</code>	At least 3 characters
Max Length	<code>max(50)</code>	At most 50 characters

Valid Examples:

- `"pass123"` ✅
- `"MySecurePassword123!"` ✅
- `"ab"` ❌ (too short)
- `"a".repeat(51)` ❌ (too long)

⚠️ **SECURITY NOTE:** Minimum 3 characters is too weak! Should be at least 8-12 characters.

4. age Field

```
age: Joi.number().required().min(1).max(90)
```

Validation	Rule	Example
Type	<code>number()</code>	Must be a number
Required	<code>required()</code>	Field must be present
Min Value	<code>min(1)</code>	At least 1
Max Value	<code>max(90)</code>	At most 90

Valid Examples:

- 25 ✓
- "25" ✓ (Joi converts to number)
- 1 ✓
- 90 ✓
- 0 ✗ (below minimum)
- 91 ✗ (above maximum)
- "twenty" ✗ (not a number)

Type Coercion:

```
// Input: { age: "25" }  
// Joi converts to: { age: 25 }
```

5. isMarried Field

```
isMarried: Joi.boolean().optional()
```

Validation	Rule	Example
Type	<code>boolean()</code>	Must be a boolean
Required	<code>optional()</code>	Field can be omitted

Valid Examples:

- true ✓
- false ✓
- "true" ✓ (Joi converts to boolean)
- "false" ✓ (Joi converts to boolean)

- `undefined`  (optional)
 - `"yes"`  (invalid boolean)
 - `1`  (not a boolean)
-

Request-Response Flow

Successful Registration Flow

1. Client sends POST request
 - ↓
 - POST /api/v1/users/register
 - Headers: { "Content-Type": "application/json" }
 - Body: {
 - "name": "John Doe",
 - "email": "john@example.com",
 - "password": "securepass123",
 - "age": 25,
 - "isMarried": false
2. Express Router receives request
 - ↓
 - Matches route: router.post("/register", ...)
3. First Middleware: validateBody(userRegisterSchema)
 - ↓
 - Validates all fields against Joi schema
 - Converts data types (string "25" → number 25)
 - Sanitizes data (trims whitespace)
 - All validations pass 
 - Calls next()
4. Second Middleware: register controller
 - ↓
 - Extracts validated data from req.body
 - Calls UserModel.create()
5. Database Operation
 - ↓
 - MongoDB creates new user document
 - Generates unique _id
 - Saves to users collection
 - Returns created document
6. Controller sends response
 - ↓
 - Status: 201 Created
 - Body: {
 - "success": true,
 - "message": "User registered successfully",
 - "data": {
 - "_id": "507f1f77bcf86cd799439011",
 - "name": "John Doe",
 - "email": "john@example.com",
 - "age": 25,
 - "isMarried": false,
 - "password": "securepass123",
 - "createdAt": "2024-01-15T10:30:00.000Z",
 - "updatedAt": "2024-01-15T10:30:00.000Z"
7. Client receives response
 - ↓
 - Success! User registered.

Failed Validation Flow

1. Client sends POST request with invalid data
 - ↓
 - POST /api/v1/users/register
 - Body: {
 - "name": "Jo", // Too short (min 3)
 - "email": "invalid", // Invalid email format
 - "password": "ab", // Too short (min 3)
 - "age": 150 // Too high (max 90)
 - // Missing: isMarried (but optional, so OK)
 - }
2. Express Router receives request
 - ↓
 - Matches route: router.post("/register", ...)
3. First Middleware: validateBody(userRegisterSchema)
 - ↓
 - Validates all fields
 - Multiple validation errors found ❌
 - error.details = [
 - { message: "name length must be at least 3 characters long" },
 - { message: "email must be a valid email" },
 - { message: "password length must be at least 3 characters long" },
 - { message: "age must be less than or equal to 90" }
 -]
 - Combines error messages
 - Throws ErrorResponse(message, 400)
4. Error Handling Middleware (not shown in code)
 - ↓
 - Catches ErrorResponse
 - Sends error response to client
5. Client receives error response
 - ↓
 - Status: 400 Bad Request
 - Body: {
 - "success": false,
 - "message": "name length must be at least 3 characters long, email must be a valid email, password length must be at least 3 characters long, age must be less than or equal to 90"
 - }
6. Registration STOPPED - user not created

Validation Rules

Complete Validation Matrix

Field	Type	Required	Min	Max	Format	Default
name	String	No (Optional)	3 chars	49 chars	Any string	undefined
email	String	Yes	-	-	Valid email	-
password	String	Yes	3 chars	50 chars	Any string	-
age	Number	Yes	1	90	Integer/Float	-

Field	Type	Required	Min	Max	Format	Default
isMarried	Boolean	No (Optional)	-	-	true/false	undefined

Validation Examples

✓ Valid Requests

Example 1: Complete Request

```
{
  "name": "John Doe",
  "email": "john@example.com",
  "password": "securePassword123",
  "age": 30,
  "isMarried": true
}
```

Result: ✓ All validations pass

Example 2: Minimal Request (optional fields omitted)

```
{
  "email": "jane@example.com",
  "password": "pass123",
  "age": 25
}
```

Result: ✓ Valid (name and isMarried are optional)

Example 3: Boundary Values

```
{
  "name": "Ann",
  "email": "a@b.co",
  "password": "abc",
  "age": 1,
  "isMarried": false
}
```

Result: ✓ Valid (minimum values accepted)

✗ Invalid Requests

Example 1: Multiple Validation Errors

```
{
  "name": "Jo",
  "email": "not-an-email",
  "password": "ab",
  "age": 150
}
```

Errors:

- name: Too short (min 3)
- email: Invalid format
- password: Too short (min 3)
- age: Too high (max 90)

Example 2: Missing Required Fields

```
{
  "name": "John Doe"
}
```

Errors:

- email: Required field missing
- password: Required field missing
- age: Required field missing

Example 3: Wrong Data Types

```
{
  "name": 12345,
  "email": "john@example.com",
  "password": "pass123",
  "age": "twenty-five",
  "isMarried": "yes"
}
```

Errors:

- name: Must be string, got number
- age: Must be number, got string
- isMarried: Must be boolean, got string

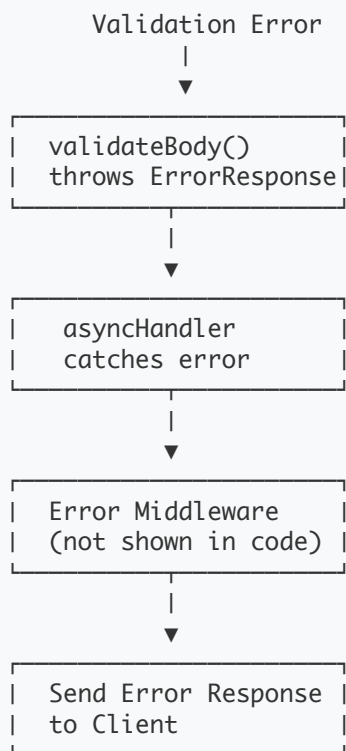
Example 4: Extra/Unknown Fields

```
{
  "name": "John",
  "email": "john@example.com",
  "password": "pass123",
  "age": 25,
  "hackerField": "malicious code"
}
```

Result:  Valid (Joi strips unknown fields by default) **After validation:** `hackerField` is removed

Error Handling

Error Flow Architecture



Error Types and Responses

1. Validation Errors (400 Bad Request)

Trigger: Invalid data format, missing required fields, constraint violations

Example Error Response:

```
{
  "success": false,
  "statusCode": 400,
  "message": "email must be a valid email, age must be less than or equal to 90"
}
```

Common Validation Errors:

- "name" length must be at least 3 characters long
- "email" must be a valid email
- "email" is required
- "password" length must be at least 3 characters long
- "age" must be a number
- "age" must be less than or equal to 90
- "age" is required
- "isMarried" must be a boolean

2. Database Errors (500 Internal Server Error)

Trigger: Database connection issues, duplicate email, schema validation errors

Example: Duplicate Email

```
{
  "success": false,
  "statusCode": 500,
  "message": "E11000 duplicate key error collection: users index: email_1"
}
```

Common Database Errors:

- Duplicate email (if unique constraint exists)
- Database connection timeout
- Mongoose validation errors
- Write operation failed

3. Server Errors (500 Internal Server Error)

Trigger: Unexpected errors, bugs in code, missing dependencies

Example:

```
{
  "success": false,
  "statusCode": 500,
  "message": "Internal Server Error"
}
```

Error Handling Best Practices (Currently Missing)

The code should implement:

```
// Global Error Handler Middleware (should be added)
app.use((err, req, res, next) => {
  let statusCode = err.statusCode || 500;
  let message = err.message || "Internal Server Error";

  // Don't leak error details in production
  if (process.env.NODE_ENV === "production" && statusCode === 500) {
    message = "Internal Server Error";
  }

  res.status(statusCode).json({
    success: false,
    statusCode,
    message,
    ...(process.env.NODE_ENV === "development" && { stack: err.stack })
  });
});
```

Security Considerations

● CRITICAL SECURITY ISSUES

1. Plain Text Password Storage

Current Code:

```
let newUser = await UserModel.create({
  name,
  age,
  email,
  isMarried,
  password /* : hashedPassword, */, // ⚠️ STORING PLAIN TEXT!
});
```

Problem: Passwords are stored in plain text in the database

Risk Level: ● CRITICAL

Impact:

- If database is compromised, all passwords are exposed
- Users who reuse passwords on other sites are at risk
- Violates security best practices and compliance requirements (GDPR, PCI DSS)

Solution: Hash passwords before storing

```
import bcrypt from "bcrypt";

// In controller, before creating user:
const hashedPassword = await bcrypt.hash(password, 10);

let newUser = await UserModel.create({
  name,
  age,
  email,
  isMarried,
  password: hashedPassword, // ✅ Store hashed password
});
```

Or better: Use Mongoose pre-save hook

```
// In User model
userSchema.pre("save", async function(next) {
  if (!this.isModified("password")) return next();
  this.password = await bcrypt.hash(this.password, 10);
  next();
});
```

2. Weak Password Requirements

Current Validation:

```
password: Joi.string().required().min(3).max(50)
```

Problem: 3 characters is extremely weak

Risk Level: 🚫 HIGH

Examples of accepted weak passwords:

- "abc" ✅ Accepted (3 chars)
- "123" ✅ Accepted (3 chars)
- "aaa" ✅ Accepted (3 chars)

Solution: Implement strong password policy

```
password: Joi.string()
  .required()
  .min(8) // Minimum 8 characters
  .max(50)
  .pattern(new RegExp("^(?=.*[a-z])(?=.*[A-Z])(?=.*\\d)(?=.*[@$!%*?&])[A-Za-z\\d@$!%*?&]{8,}$"))
  .messages({
    "string.pattern.base": "Password must contain at least one uppercase letter, one lowercase letter, one number, and one special character"
  })
```


Password Requirements:

-  Minimum 8 characters
-  At least one uppercase letter
-  At least one lowercase letter
-  At least one number
-  At least one special character (@\$!%*?&)

3. Password Exposure in Response**Current Code:**

```
res.status(201).json({
  success: true,
  message: "User registered successfully",
  data: newUser, // ⚠️ Contains password!
});
```

Problem: Password (even if hashed) is sent back to client

Risk Level:  MEDIUM

Solution: Exclude password from response

```
// Option 1: Manually delete password
delete newUser._doc.password;

// Option 2: Use Mongoose select
let newUser = await UserModel.create({...}).select("-password");

// Option 3: Transform in model
userSchema.set("toJSON", {
  transform: function(doc, ret) {
    delete ret.password;
    return ret;
  }
});

res.status(201).json({
  success: true,
  message: "User registered successfully",
  data: newUser, // ✅ Password excluded
});
```

4. No Email Uniqueness Check

Current Issue: No validation for duplicate emails

Problem: Multiple users can register with same email

Risk Level:  MEDIUM

Solution: Add unique constraint in Mongoose model

```
// In User model
const userSchema = new mongoose.Schema({
  email: {
    type: String,
    required: true,
    unique: true, // ✅ Prevent duplicates
    lowercase: true,
    trim: true
  }
});
```

Better: Check before creating

```
const existingUser = await UserModel.findOne({ email });
if (existingUser) {
  throw new ErrorResponse("Email already registered", 409);
}
```

5. No Rate Limiting

Current Issue: No limit on registration attempts

Problem: Vulnerable to:

- Spam registrations
- DDoS attacks
- Automated bot registrations

Risk Level: 🟡 MEDIUM

Solution: Implement rate limiting

```
import rateLimit from "express-rate-limit";

const registerLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 5, // 5 requests per windowMs
  message: "Too many registration attempts, please try again later"
});

router.post("/register", registerLimiter, validateBody(userRegisterSchema), register);
```

6. No Input Sanitization

Current Issue: No protection against XSS or injection attacks

Problem: Malicious input could be stored and executed

Example Attack:

```
{
  "name": "<script>alert('XSS')</script>",
  "email": "hacker@evil.com",
  "password": "pass123",
  "age": 25
}
```

Solution: Sanitize inputs

```
import mongooseSanitize from "express-mongo-sanitize";
import xss from "xss-clean";

app.use(mongooseSanitize()); // Prevent NoSQL injection
app.use(xss()); // Prevent XSS attacks
```

Security Checklist

Security Measure	Status	Priority	Notes
Password Hashing	✗ Missing	🔴 Critical	Store bcrypt hash, not plain text
Strong Password Policy	✗ Weak	🔴 High	Min 3 chars is too weak
Hide Password in Response	✗ Missing	🟡 Medium	Don't send password back
Email Uniqueness	✗ Missing	🟡 Medium	Prevent duplicate accounts
Rate Limiting	✗ Missing	🟡 Medium	Prevent spam/DDoS
Input Sanitization	✗ Missing	🟡 Medium	Prevent XSS/injection
HTTPS Only	? Unknown	🔴 High	Use SSL/TLS
JWT/Sessions	? Unknown	🔴 High	For authentication
CORS Configuration	? Unknown	🟡 Medium	Restrict origins
Helmet Security Headers	? Unknown	🟡 Medium	Add security headers

Example Requests

Using cURL

Successful Registration

```
curl -X POST http://localhost:9000/api/v1/users/register \
-H "Content-Type: application/json" \
-d '{
  "name": "John Doe",
  "email": "john@example.com",
  "password": "securePass123",
  "age": 30,
  "isMarried": true
}'
```

Response (201 Created):

```
{
  "success": true,
  "message": "User registered successfully",
  "data": {
    "_id": "507f1f77bcf86cd799439011",
    "name": "John Doe",
    "email": "john@example.com",
    "age": 30,
    "isMarried": true,
    "password": "securePass123",
    "createdAt": "2024-01-15T10:30:00.000Z",
    "updatedAt": "2024-01-15T10:30:00.000Z",
    "__v": 0
  }
}
```

Failed Validation

```
curl -X POST http://localhost:9000/api/v1/users/register \
-H "Content-Type: application/json" \
-d '{
  "name": "Jo",
  "email": "invalid-email",
  "password": "ab",
  "age": 150
}'
```

Response (400 Bad Request):

```
{
  "success": false,
  "statusCode": 400,
  "message": "\"name\" length must be at least 3 characters long, \"email\" must be a valid email, \"password\" length must be at least 3 characters long, \"age\" must be less than or equal to 90"
}
```

Using JavaScript Fetch API

```
// Successful registration
async function registerUser() {
  try {
    const response = await fetch("http://localhost:9000/api/v1/users/register", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify({
        name: "Jane Smith",
        email: "jane@example.com",
        password: "myPassword123",
        age: 28,
        isMarried: false
      })
    });

    const data = await response.json();

    if (response.ok) {
      console.log("Registration successful:", data);
      // Handle success (e.g., redirect to login page)
    } else {
      console.error("Registration failed:", data.message);
      // Display error to user
    }
  } catch (error) {
    console.error("Network error:", error);
  }
}

registerUser();
```

Using Axios

```
import axios from "axios";

// Successful registration
async function registerUser() {
  try {
    const response = await axios.post(
      "http://localhost:9000/api/v1/users/register",
      {
        name: "Bob Johnson",
        email: "bob@example.com",
        password: "bobPass456",
        age: 35,
        isMarried: true
      }
    );

    console.log("Success:", response.data);
  } catch (error) {
    if (error.response) {
      // Server responded with error status
      console.error("Error:", error.response.data.message);
    } else if (error.request) {
      // Request made but no response
      console.error("No response from server");
    } else {
      // Something else happened
      console.error("Error:", error.message);
    }
  }
}

registerUser();
```

Using Postman

Setup:

1. Method: **POST**
2. URL: **http://localhost:9000/api/v1/users/register**
3. Headers:
 - Key: **Content-Type**, Value: **application/json**
4. Body (raw JSON):

```
{
  "name": "Alice Wonder",
  "email": "alice@example.com",
  "password": "alicePass789",
  "age": 26,
  "isMarried": false
}
```

Complete Flow Summary

Registration Success Path

```

Client Request
↓
[Express App: app.use("/api/v1/users", userRoutes)]
↓
[Router: router.post("/register", ...)]
↓
[Middleware 1: validateBody(userRegisterSchema)]
  ↳ Validate request body against Joi schema
  ↳ Check all required fields present
  ↳ Check data types correct
  ↳ Check constraints (min, max, format)
  ↳ Convert types (string → number)
  ↳ Sanitize data (trim whitespace)
  ↳ All valid ✅ → call next()
↓
[Middleware 2: register controller]
  ↳ Extract validated data from req.body
  ↳ Destructure: {name, age, email, isMarried, password}
  ↳ Call UserModel.create(userData)
↓
[Database: MongoDB]
  ↳ Create new document
  ↳ Generate unique _id
  ↳ Add timestamps (createdAt, updatedAt)
  ↳ Save to users collection
  ↳ Return saved document
↓
[Controller: Format response]
  ↳ Status: 201 (Created)
  ↳ success: true
  ↳ message: "User registered successfully"
  ↳ data: newUser object
↓
[Client receives response]
  ↳ User successfully registered! 🎉

```

Registration Failure Path (Validation Error)

```

Client Request (invalid data)
  ↓
[Express App → Router]
  ↓
[Middleware 1: validateBody]
  ↳ Validate request body
  ↳ Find validation errors ❌
  ↳ Extract error messages
  ↳ Combine: "error1, error2, error3"
  ↳ throw new ErrorResponse(message, 400)
  ↓
[asyncHandler catches error]
  ↳ Pass to error handler: next(error)
  ↓
[Error Handling Middleware]
  ↳ Extract statusCode (400)
  ↳ Extract message
  ↳ Format error response
  ↓
[Client receives error response]
  ↳ Status: 400, success: false, message: "..." ❌

```

Recommended Improvements

1. Hash Passwords

```

import bcrypt from "bcrypt";

const hashedPassword = await bcrypt.hash(password, 10);
let newUser = await UserModel.create({
  name,
  age,
  email,
  isMarried,
  password: hashedPassword
});

```

2. Exclude Password from Response

```

newUser = newUser.toObject();
delete newUser.password;

res.status(201).json({
  success: true,
  message: "User registered successfully",
  data: newUser
});

```


3. Check for Existing Email

```
const existingUser = await UserModel.findOne({ email });
if (existingUser) {
  throw new ErrorResponse("Email already registered", 409);
}
```

4. Stronger Password Validation

```
password: Joi.string()
  .required()
  .min(8)
  .max(50)
  .pattern(/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])$/)
```

5. Add Rate Limiting

```
import rateLimit from "express-rate-limit";

const registerLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 5
});

router.post("/register", registerLimiter, validateBody(userRegisterSchema), register);
```

6. Input Sanitization

```
import mongoSanitize from "express-mongo-sanitize";
import xss from "xss-clean";

app.use(mongoSanitize());
app.use(xss());
```

7. Email Verification

```
// Send verification email after registration
await sendVerificationEmail(newUser.email, verificationToken);
```

8. Better Error Messages

```
// Custom Joi messages
email: Joi.string()
  .email()
  .required()
  .messages({
    "string.email": "Please provide a valid email address",
    "any.required": "Email is required"
  })
```

Technology Dependencies

Required NPM Packages

```
{
  "dependencies": {
    "express": "^4.18.2",
    "joi": "^17.9.2",
    "mongoose": "^7.3.1"
  },
  "devDependencies": {
    "nodemon": "^3.0.1"
  }
}
```

Recommended Additional Packages

```
{
  "dependencies": {
    "bcrypt": "^5.1.0",           // Password hashing
    "express-rate-limit": "^6.8.1", // Rate limiting
    "express-mongo-sanitize": "^2.2.0", // NoSQL injection prevention
    "xss-clean": "^0.1.4",        // XSS protection
    "helmet": "^7.0.0",           // Security headers
    "cors": "^2.8.5",             // CORS configuration
    "dotenv": "^16.3.1"           // Environment variables
  }
}
```

Conclusion

This user registration system provides a solid foundation with:

-  RESTful API design
-  Input validation with Joi
-  Async error handling

-  Database integration

However, it requires critical security improvements:

-  Hash passwords before storage
-  Strengthen password requirements
-  Implement rate limiting
-  Add email uniqueness validation
-  Exclude passwords from responses
-  Add input sanitization

Priority Actions:

1. Implement password hashing (CRITICAL)
 2. Strengthen password validation (HIGH)
 3. Add email uniqueness check (MEDIUM)
 4. Implement rate limiting (MEDIUM)
 5. Add input sanitization (MEDIUM)
-

Additional Resources

- [Joi Documentation](#)
 - [Express.js Guide](#)
 - [Mongoose Documentation](#)
 - [bcrypt Documentation](#)
 - [OWASP Password Guidelines](#)
-

License & Author

Created by: Avinash **Date:** January 2025 **Phone:** +91-6204732828 **Github:** <https://github.com/avinash-25>
